```python
planation_module.py
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
import torch

# Load the improved model for explanations
explain_tokenizer = AutoTokenizer.from_pretrained("MBZUAI/LaMini-Flan-T5-783M")
explain_model = AutoModelForSeq2SeqLM.from_pretrained("MBZUAI/LaMini-Flan-T5-783M")

def explain_topic(topic: str) -> str:
    input_text = f"Explain the concept of '{topic}' in a simple and clear way for a school student."
    inputs = explain_tokenizer(input_text, return_tensors="pt")

    outputs = explain_model.generate(
        **inputs,
        max_new_tokens=150,
        temperature=0.7,
        top_k=50,
        top_p=0.95,
        do_sample=True
    )

    explanation = explain_tokenizer.decode(outputs[0], skip_special_tokens=True)
    return explanation
```

Fig. Explanation module logic

2. QnA Module

EduGenie is powered by Gemini 1.5 Pro, enabling it to handle a wide range of general knowledge and academic question-answering tasks with precision. Its advanced understanding and contextual capabilities make it an effective tool for learners seeking accurate, AI-driven assistance across diverse educational topics and subjects.

```python
def answer_question_with_gemini(question: str) -> str:
    try:
        model = genai.GenerativeModel(model_name="models/gemini-1.5-pro")
        response = model.generate_content(question)
        return response.text.strip()
    except Exception as e:
        return f"⚠️ Error in QnA: {e}"
```

Fig. Q/A Module

3. Quiz Module

EduGenie generates three multiple-choice questions (MCQs) from a given passage, each containing four carefully crafted options. It utilizes the Gemini model to comprehend the context and meaning of the passage, ensuring that the questions are relevant and the distractors are plausible. The final output is structured in JSON format for easy integration.

```
def clean_json_block(text):
    # Remove Markdown ```json code fences
    return re.sub(r"```(?:json)?\n(.*?)```", r"\1", text, flags=re.DOTALL).strip()

def generate_quiz(text: str) -> list:
    try:
        model = genai.GenerativeModel(model_name="models/gemini-1.5-pro")

        prompt = f"""
You are a quiz generator.

From the following passage, create 3 multiple-choice questions. Each question should include:
- A "question"
- A list of 4 "options"
- A correct "answer" that must exactly match one of the options.

Format your output as **valid JSON**, like this:
[
  {{
    "question": "What is ...?",
    "options": ["A", "B", "C", "D"],
    "answer": "A"
  }}
]

Passage:
{text}
"""
        response = model.generate_content(prompt)
        quiz_text = response.text.strip()

        # 🧼 Clean markdown code blocks if any
        cleaned_text = clean_json_block(quiz_text)
```

Fig. Quiz Module Logic

It sends a structured prompt to the model, expecting a valid JSON response with each question containing four options and a correct answer. To ensure proper parsing, it cleans any Markdown code blocks using the clean_json_blockfunction. The cleaned response is then parsed into a Python list. If an error occurs during generation or parsing, it returns a detailed error message for debugging.

4. Summary Module

This feature leverages Gemini's generative capabilities to summarize long paragraphs into concise, easy-to-understand versions, making it ideal for quick revision. It ensures the core information is retained while eliminating redundancy, helping learners absorb key points efficiently without losing clarity or context.